

What is Linux Kernel Porting?



One of the aspects of hacking a Linux kernel is to port it. While this might sound difficult, it won't be once you read this article. The author explains porting techniques in a simplified manner.

With the evolution of embedded systems, porting has become extremely important. Whenever you have new hardware at hand, the first and the most critical thing to be done is porting. For hobbyists, what has made this even more interesting is the open source nature of the Linux kernel. So, let's dive into porting and understand the nitty-gritty of it.

Porting means making something work on an environment it is not designed for. Embedded Linux porting means making Linux work on an embedded platform, for which it was not designed. Porting is a broader term and when I say 'embedded Linux porting', it not only involves Linux kernel porting, but also porting a first stage bootloader, a second stage bootloader and, last but not the least, the applications. Porting differs from development. Usually, porting doesn't involve as much of coding as in development. This means that there is already some code available and it only needs to be fine-tuned to the desired

target. There may be a need to change a few lines here and there, before it is up and running. But, the key thing to know is, *what* needs to be changed and where.

What Linux kernel porting involves

Linux kernel porting involves two things at a higher level: architecture porting and board porting. Architecture, in Linux terminology, refers to CPU. So, architecture porting means adapting the Linux kernel to the target CPU, which may be ARM, Power PC, MIPS, and so on. In addition to this, SOC porting can also be considered as part of architecture porting. As far as the Linux kernel is concerned, most of the times, you don't need to port it for architecture as this would already be supported in Linux. However, you still need to port Linux for the board and this is where the major focus lies. Architecture porting entails porting of initial start-up code, interrupt service routines, dispatcher routine, timer routine, memory management, and so on.